1. Command Model (Go)

This is the standard command envelope every action uses.

```
type WSCommand struct {
    Type      string
`json:"type"`      // COMMAND
    Action    string
`json:"action"`    //
close_positions, delete_orders
    Filters   map[string]string
`json:"filters"`   // criteria
    SessionID string
`json:"session_id"`
    Magic     int64
`json:"magic"`     // EA magic
number
}
```

2. Close ALL Open Positions

```
cmd := WSCommand{
    Type:   "COMMAND",
    Action: "CLOSE_POSITIONS",
    Filters: map[string]string{
        "scope": "ALL",
    },
    SessionID: "session-123",
    Magic:     10001,
}
```

EA logic:

- Loop through all open positions
- Close regardless of symbol, side, PnL

3. Close ONLY Profitable Positions

```
cmd := WSCommand{
    Type:   "COMMAND",
    Action: "CLOSE_POSITIONS",
    Filters: map[string]string{
        "pnl": "PROFIT",
    },
    SessionID: "session-123",
    Magic:     10001,
}
```

EA condition:

```
PositionProfit() > 0
```

4. Close ONLY Losing Positions

```
cmd := WSCommand{
    Type:   "COMMAND",
    Action: "CLOSE_POSITIONS",
    Filters: map[string]string{
        "pnl": "LOSS",
    },
    SessionID: "session-123",
```

```
    Magic:     10001,
}
```

EA condition:

```
PositionProfit() < 0
```

5. Close ONLY BUY Positions

```
cmd := WSCommand{
    Type:   "COMMAND",
    Action: "CLOSE_POSITIONS",
    Filters: map[string]string{
        "side": "BUY",
    },
    SessionID: "session-123",
    Magic:     10001,
}
```

EA condition:

```
POSITION_TYPE_BUY
```

6. Close ONLY SELL Positions

```
cmd := WSCommand{
    Type:   "COMMAND",
    Action: "CLOSE_POSITIONS",
    Filters: map[string]string{
        "side": "SELL",
    },
    SessionID: "session-123",
    Magic:     10001,
}
```

EA condition:

```
POSITION_TYPE_SELL
```

7. Delete ALL Pending Orders

```
cmd := WSCommand{
    Type:   "COMMAND",
    Action: "DELETE_ORDERS",
```

```
    Filters: map[string]string{
        "scope": "ALL",
    },
    SessionID: "session-123",
    Magic:     10001,
}
```

EA logic:

- Loop OrdersTotal()
- OrderDelete(ticket)

8. WebSocket Send (Go)

```
func sendCommand(conn
*websocket.Conn, cmd WSCommand)
error {
    return conn.WriteJSON(cmd)
}
```

9. Why This Design Works

- Stateless commands
- RMS / OMS can trigger emergency liquidation
- Magic number isolates per-EA/session
- Same protocol works for:
    - MT5 today
    - Binance / TWS tomorrow
-

10. Critical Rule (Tell the EA Dev)

EA must always:

1. Filter by Magic
2. Execute atomically (no partial close)
3. Acknowledge with status:

```
{ "status": "OK", "action":
"CLOSE_POSITIONS", "count": 12 }
```

Below is a clean, dev-ready extension to your existing WebSocket command protocol.

This adds partial close by percentage, works for profitable and losing positions, and supports per-position targeting.

Everything is Go-side, EA just executes.

1. Extended Command Model (Go)

We extend the command with PositionID and ClosePercent.

```
type WSCommand struct {
    Type          string
`json:"type"`          // COMMAND
    Action        string
`json:"action"`        //
PARTIAL_CLOSE
    SessionID     string
`json:"session_id"`
    Magic         int64
`json:"magic"`
    PositionID    string
`json:"position_id"` // MT5 ticket
(string-safe)
    ClosePercent float64
`json:"close_pct"`    // 0 < pct <=
100
    Filters       map[string]string
`json:"filters"`      // optional
}
```

2. Partial Close – Specific Position (Any PnL)

Closes X% of one open position, regardless of profit or loss.

```
cmd := WSCommand{
    Type:          "COMMAND",
    Action:        "PARTIAL_CLOSE",
    SessionID:     "session-123",
    Magic:         10001,
    PositionID:    "458921774",
    ClosePercent: 25.0, // close
25%
}
```

EA logic:

```
close_volume = position_volume *
(close_pct / 100)
```

3. Partial Close – ALL Profitable Positions (%)

```
cmd := WSCommand{
    Type:       "COMMAND",
```

```
    Action:    "PARTIAL_CLOSE",
    SessionID: "session-123",
    Magic:     10001,
    Filters: map[string]string{
        "pnl": "PROFIT",
    },
    ClosePercent: 50.0,
}
```

EA logic:

- Loop positions
- If PositionProfit() > 0
- Close 50% of each

4. Partial Close – ALL Losing Positions (%)

```
cmd := WSCommand{
    Type:      "COMMAND",
    Action:    "PARTIAL_CLOSE",
```

```
    SessionID: "session-123",
    Magic:     10001,
    Filters: map[string]string{
        "pnl": "LOSS",
    },
    ClosePercent: 30.0,
}
```

EA logic:

- If PositionProfit() < 0
- Close 30%

5. Partial Close – BUY or SELL Only (%)

BUY Only

```
cmd := WSCommand{
    Type:      "COMMAND",
    Action:    "PARTIAL_CLOSE",
    SessionID: "session-123",
    Magic:     10001,
    Filters: map[string]string{
        "side": "BUY",
    },
    ClosePercent: 40.0,
}
```

SELL Only

```
cmd := WSCommand{
    Type:      "COMMAND",
    Action:    "PARTIAL_CLOSE",
    SessionID: "session-123",
    Magic:     10001,
    Filters: map[string]string{
        "side": "SELL",
    },
    ClosePercent: 40.0,
}
```

6. Safety Rules (MANDATORY – Give to EA Dev)

EA must enforce:

1. $0 < \text{ClosePercent} \leq 100$
2. Resulting volume ≥ broker minimum lot
3. Volume rounded to symbol step
4. If ClosePercent == 100 → full close
5. Filter by Magic always
6. Reject if position already closing

7. WebSocket Send Function (Go)

```
func sendCommand(conn
*websocket.Conn, cmd WSCommand)
error {
    return conn.WriteJSON(cmd)
}
```

8. EA Acknowledgement (Required)

EA must respond:

```
{
  "status": "OK",
  "action": "PARTIAL_CLOSE",
  "position_id": "458921774",
  "closed_pct": 25,
  "remaining_volume": 0.75
}
```

On failure:

```
{
  "status": "ERROR",
```

```
  "reason": "MIN_LOT_VIOLATION"
}
```